

LKD4DyTAG: LLM-Driven Knowledge Distillation for Dynamic Text-Attributed Graphs

LKD4DyTAG (2024) introduces a lightweight graph neural network architecture for dynamic graphs with rich text on nodes and edges. The model leverages a large language model (LLM) during training only — not inference — to guide a spatio-temporal GNN in learning semantically informed edge representations. The key idea is to distill knowledge from LLMs into a time-aware GNN for tasks like future link prediction and edge classification.

Temporal Edge Encoding (With Examples)

To model temporal dynamics, each edge e_{uv} is assigned a **temporal mask vector** indicating when the edge becomes active. For example, if an edge appears at time step 3 in a graph with 10 time steps, the mask is:

[0, 0, 0, 1, 1, 1, 1, 1, 1, 1]

Where:

- 0 = edge not yet present
- 1 = edge is active
- -1 = future (masked during training)

This mask is concatenated with the BERT-based edge embedding to form the **temporal edge representation**, which allows the GNN to know when this edge should contribute to message passing.

Spatio-Temporal Graph Encoding (With Examples)

Each node and edge has associated text (e.g., user profiles, messages, reviews), which are encoded using a **frozen BERT encoder**.

Example:

- Node u (Alice): "Software engineer focused on distributed systems"
- Node v (Bob): "Cloud architect interested in Kubernetes and scalability"

- Edge: "Alice commented on Bob's architecture post: 'You should look into service mesh patterns.'"

These are encoded into vectors:

- $h_u, h_v = \text{BERT}(\text{node text})$
- $h_{\text{edge}} = \text{BERT}(\text{edge text})$

The **temporal mask** is then concatenated with h_{edge} , and the resulting vector is used in the GNN's message passing. After updating node embeddings, the final edge embedding is computed via a **Hadamard product**:

$$h_{uv} = h_u \odot h_{vh_{\{uv\}}} = h_u \circledast h_v$$

This embedding serves as the student GNN's representation of the edge.

LLM-Based Textual Edge Encoding (With Examples)

For each edge, the model constructs a **prompt** that includes:

- Node u's text
- Node v's text
- Edge text
- Text from neighboring nodes
- A binary flag indicating whether this is a real or fake edge

Example prompt:

User A: Alice is a backend engineer with experience in gRPC and service discovery.

User B: Bob asked a question about microservice scaling.

Interaction: "Have you explored Istio or Linkerd?"

Alice also replied to posts on API gateways and load balancing.

Label: Real

This prompt is passed to a frozen LLM (e.g., LLaMA2 or GPT-3.5), which outputs a **high-dimensional edge embedding**, serving as the **teacher signal**.

Knowledge Distillation and Losses

The GNN is trained to match the LLM's embedding via **cosine similarity**:

$$L_{KD} = \exp(-\cos(h_{\text{textual}}, h_{\text{spatio-temporal}})) \quad \mathcal{L}_{\text{KD}} = \exp(-\cos(h_{\text{textual}}, h_{\text{spatio-temporal}}))$$

The student GNN is also trained on two supervised tasks:

- $\mathcal{L}_{\text{flp}}$: **Future link prediction** — will the edge appear in the next time step?
- $\mathcal{L}_{\text{ec}}$: **Edge classification** — what type of interaction is this?

The total training loss is a weighted combination:

$$\mathcal{L} = \lambda_{\text{flp}} \mathcal{L}_{\text{flp}} + \lambda_{\text{ec}} \mathcal{L}_{\text{ec}} + \lambda_{\text{KD}} L_{\text{KD}} \quad \mathcal{L} = \lambda_{\text{flp}} \mathcal{L}_{\text{flp}} + \lambda_{\text{ec}} \mathcal{L}_{\text{ec}} + \lambda_{\text{KD}} \mathcal{L}_{\text{KD}}$$

This forces the GNN to be both **task-effective** and **LLM-aligned**.

Evaluation and Results

LKD4DyTAG was evaluated on 6 real-world dynamic text-attributed graph datasets, including:

- Enron (emails)
- StackOverflow (posts)
- GDELT (geopolitical events)

It outperformed dynamic GNN baselines (e.g., DyRep, TGAT, CAWN) on both edge classification and future link prediction tasks. Ablation studies confirmed that knowledge distillation and the temporal mask both significantly contribute to performance.

Why It Matters for the Survey

LKD4DyTAG is one of the first models to combine:

- **LLM-based textual supervision**
- **Temporal edge encoding**
- **Graph structure learning**
in a **distillation-based training regime**. Unlike models that use LLMs at inference time,

it trains a lightweight GNN to inherit LLM-style semantic reasoning, making it scalable and efficient for deployment on dynamic, text-rich graphs.